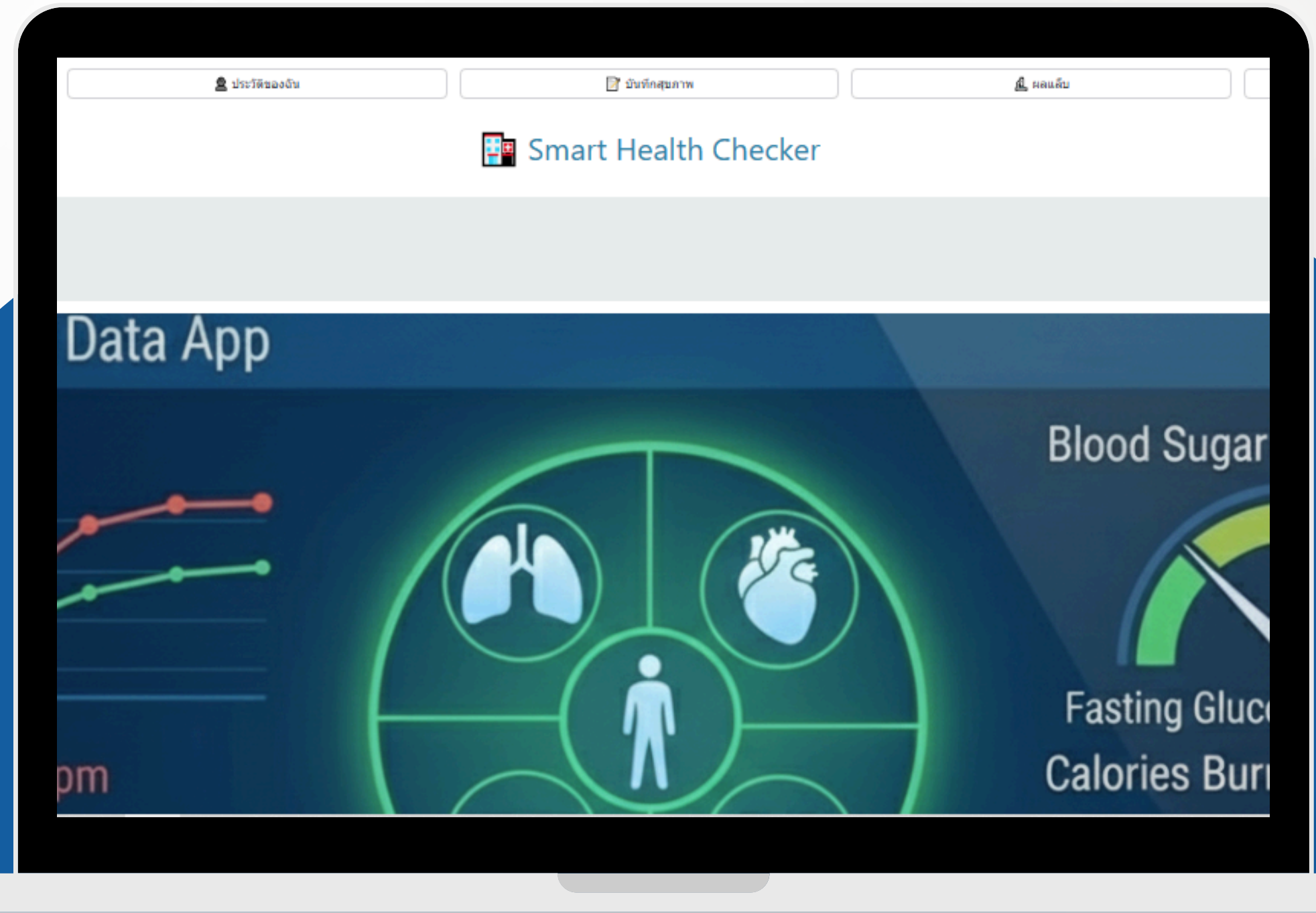


Smart Health Checker

Data Product



Personalized Data Product for Health Risk Assessment

Smart Health Checker

▶▶▶	Issues	01
▶▶▶	Motivation	02
▶▶▶	Objective	03
▶▶▶	System Overview	04
▶▶▶	Next Step	08
▶▶▶	Appendix	09
	Coding :System Deployment	



ISSUES



มีผลตรวจสุขภาพ แต่ไม่สามารถอ่านผลสุขภาพ
และบอกความเสี่ยงที่จะเกิดโรคต่างๆได้



ไม่มีเครื่องมือ Preventive Health
แบบ personalize



การตีความยังต้องพึ่งแพทย์ทั้งหมด
สามารถเข้าถึงได้ยาก และมีค่าใช้จ่ายสูง

Motivation

เข้าใจสุขภาพของตัวเองทันที จากการ
แปลผล Lab เป็นภาษาที่ง่าย



ได้รับคำแนะนำเฉพาะบุคคล (Personalized
Recommendation)



เห็นภาพรวม และแนวโน้มสุขภาพ ผ่าน
Dashboard แบบเรียลไทม์



รู้ความเสี่ยงโรคส่วนบุคคล โดยใช้
ข้อมูลจากหลายปัจจัย



รู้ระดับความเสี่ยงของสุขภาพ ทำให้รู้ได้ว่า
ควรไปพบแพทย์เมื่อไร



Objectives

01

สร้างระบบช่วยผู้ใช้ประเมินความเสี่ยงโรคพื้นฐานแบบ Real-time ได้แก่ 5 โรค เบาหวาน, ความดัน, โรคหัวใจ, โรคไต ,โรคตับ

02

รวมข้อมูลสุขภาพหลายมิติ เช่น ค่า Lab /ประวัติส่วนตัว / พฤติกรรมการใช้ชีวิต เข้าด้วยกัน

03

ใช้ทั้ง Machine Learning + Clinical rules เพื่อเพิ่มความแม่นยำและความน่าเชื่อถือของ Risk scoring แบบรายบุคคล

04

ทำ Dashboard ประเมินผลสุขภาพ และใช้ AI เพื่ออธิบายผลในภาษาที่คนทั่วไปเข้าใจได้ง่าย

05

สร้างต้นแบบของระบบติดตามสุขภาพเชิงรุกที่สามารถขยายต่อยอดไปสู่การใช้งานจริงในอนาคต

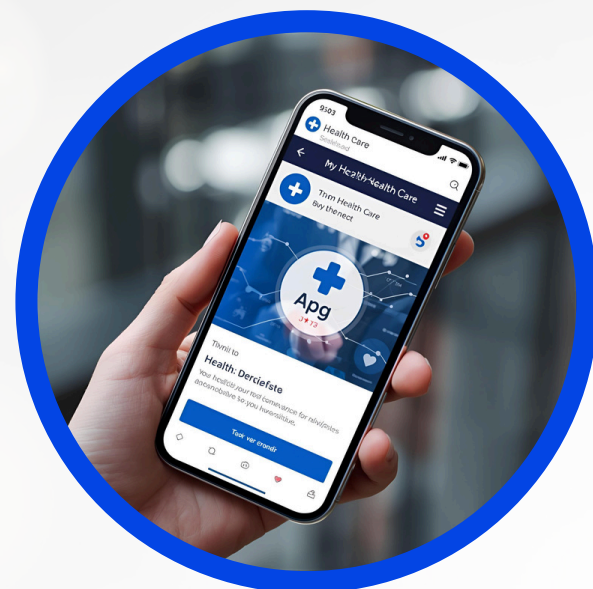
HELLO, SMART HEALTH CHECKER

Solution by Traditional and Ai add-on

SYSTEM OVERVIEW

ภาพรวมระบบ

Application มีการทำงาน
ทั้งหมด 3 ส่วนหลัก



Part 1: Create Your Personalized Profile : Input Your Health Data

การลงทะเบียนผู้ใช้งาน และรับข้อมูลเกี่ยวกับสุขภาพ
เพื่อประเมินความเสี่ยงของโรคเบาหวาน , ความดันสูง , โรคหัวใจ , โรค
ไต, โรคตับและโรคอ้วนได้



Part 2: Disease Risk Assessment System +AI Assistant

ระบบสามารถประเมินความเสี่ยงของโรคดังกล่าวและสรุปผล
สุขภาพผู้ใช้งาน โดยสามารถวิเคราะห์ "ระดับความเสี่ยงการเกิดโรค
และอธิบายโรค อีกทั้งยังมีคำแนะนำจาก AI" อีกด้วย



Part 3: Historical Health and Visualization System

ระบบสามารถเก็บข้อมูลสุขภาพของผู้ใช้งาน เพื่อให้สามารถติดตาม
สุขภาพในอดีตและปัจจุบัน และแสดงผลด้วย Visualization แนว
โน้มสุขภาพของผู้ใช้งานและข้อมูลสุขภาพเชิงลึกได้

Part 1: Create Your Personalized Profile : Input Your Health Data

INPUT YOUR HEALTH DATA

1. **Demographics** — อายุ เพศ BMI
2. **Lifestyle** — การสูบบุหรี่ ความดัน ประวัติครอบครัว
3. **Lab Results** — Glucose, HbA1c, Cholesterol, Creatinine, GFR และอื่นๆ

RISK ASSESSMENT AND PERSONALIZED HEALTH SUMMARY

ระบบจะประเมินค่าผล
ตรวจสุขภาพเบื้องต้น
และผลของน้ำตาลใน
เลือด ค่าตับ และไต
ต่างๆ ซึ่งระบุได้ว่าอยู่
ในเกณฑ์ที่ปกติหรือไม่

แบบฟอร์มบันทึกสุขภาพ

วันที่บันทึกข้อมูล
2025/12/05

ข้อมูลพื้นฐาน

เพศ	น้ำหนัก (kg)	BMI
Female	55.00	21.48
อายุ	ส่วนสูง (cm)	สูบบุหรี่
27	160.00	No

โรคประจำตัว

ระดับน้ำตาลกลูโคส (mg/dL)	เบาหวาน	ไขมันลดความดัน
145.00	No	No
ระดับน้ำตาลสะสม HbA1c	โรคหัวใจ	
7.50	No	

พฤติกรรมกรกิน

ทานอาหารแคลอรีสูงบ่อย	มื้ออาหารต่อวัน
No	1
กินผักผลไม้อาหาร	กินของว่างระหว่างมื้อ

เลือกวันที่แสดงผล
03/12/2025

ข้อมูลสุขภาพพื้นฐาน

ดัชนีมวลกาย (BMI)	22.5	ปกติ
ค่าปกติ: 18.5-24.9		
ความดันโลหิต	150.0/80.0 mmHg	สูง
ค่าปกติ: ≤ 120/80 mmHg		
อัตราการเต้นหัวใจ	65 bpm	ปกติ
ค่าปกติ: 60-100 bpm		
ระดับน้ำตาลในเลือด		
น้ำตาลกลูโคส	145.0 mg/dL	สูง
ค่าปกติ: ≤ 100 mg/dL		
HbA1c	7.5 %	สูง

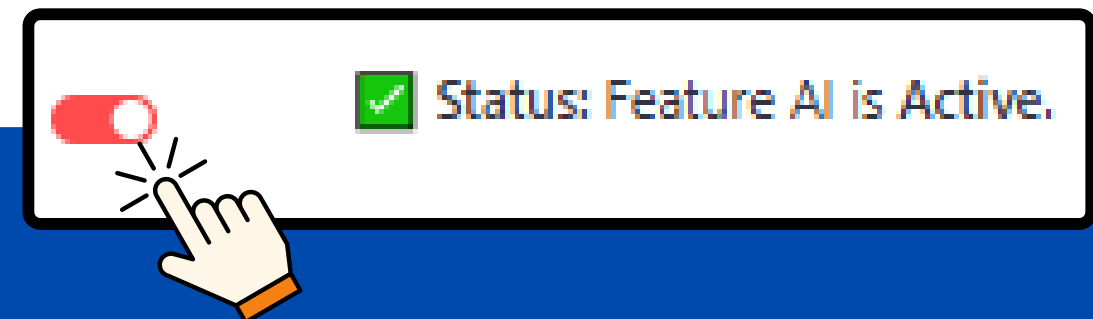
Part 2: Disease Risk Assessment System +AI Assistant

1

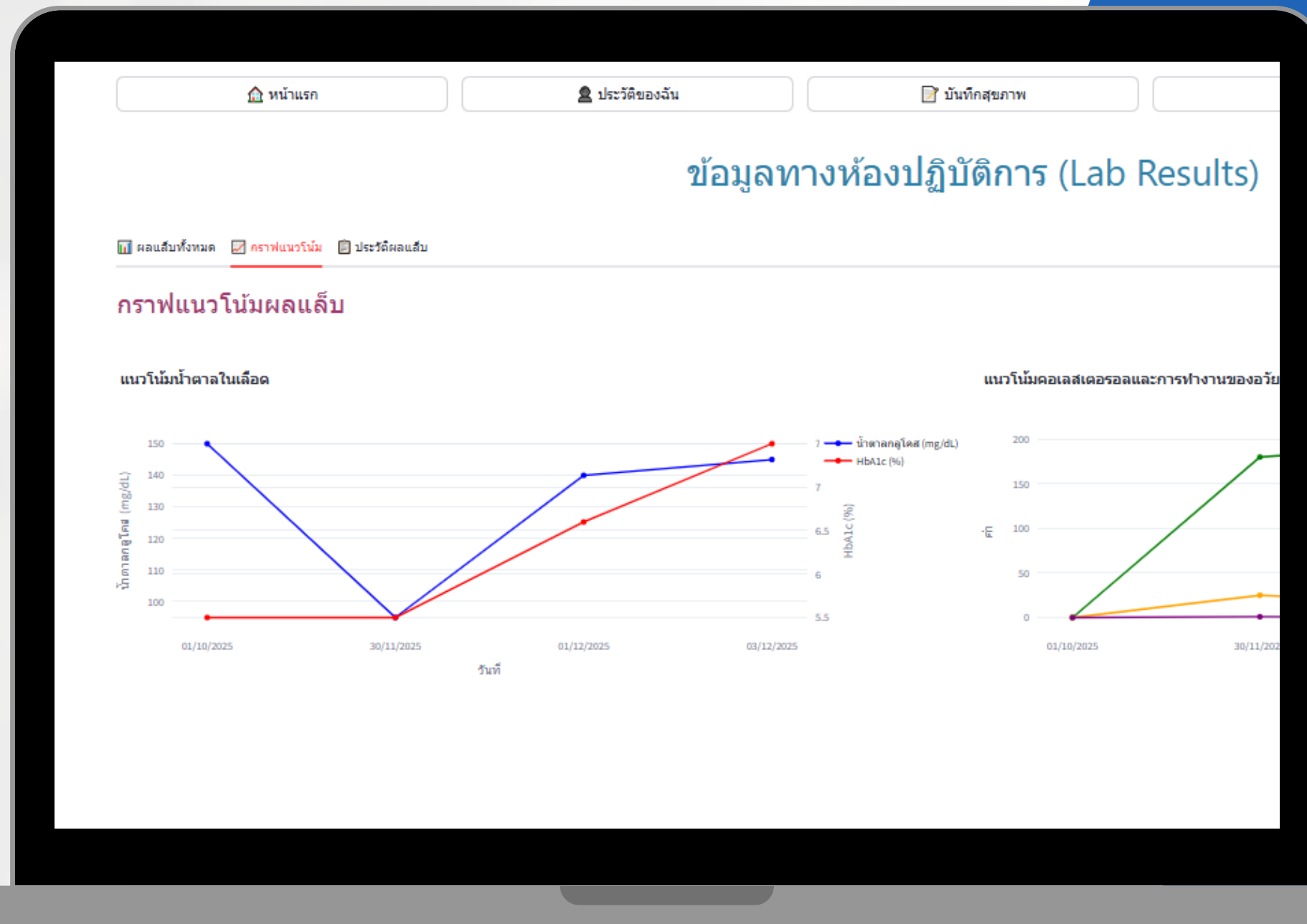
ประเมินความเสี่ยงของโรคและสรุปผล
สุขภาพ ระดับความเสี่ยงการเกิดโรค

2

พร้อมด้วยการวิเคราะห์การเกิดโรค และ
คำแนะนำที่เหมาะสมในการดูแลสุขภาพ
โดย AI

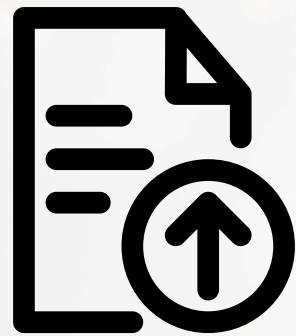


Part 3: Historical Health and Visualization System



เก็บข้อมูลสุขภาพของผู้ใช้งาน เพื่อ
ให้สามารถติดตามสุขภาพในอดีตและ
ปัจจุบัน
และแสดงผลด้วย Visualization
แนวโน้มสุขภาพของผู้ใช้งานและ
ข้อมูลสุขภาพเชิงลึกได้

OUR NEXT STEP



01 Lab Results Upload

เพิ่มฟังก์ชันการอัปโหลดเอกสารผลตรวจ
สุขภาพ (PDF, ภาพถ่าย) เพื่อดึงข้อมูลตัวเลข
ผลแล็บเข้าสู่ระบบโดยอัตโนมัติ



02 Integration with Wearable Devices (API)

เชื่อมต่อระบบกับ API ของอุปกรณ์สวมใส่ เช่น
Smart Watch เพื่อดึงข้อมูลแบบ Real-time
เช่น อัตราการเต้นของหัวใจ, การเผาผลาญแคลอรี,
และรูปแบบการนอนหลับ



Appendix

Coding :System Deployment

Streamlit UI (Multi-pages)

DuckDB local

External DB (MongoDB/PostgreSQL)

Python Backend + ML(Traditional)

AI Add on

Cache + Resource caching

Multi-Page

```
def navigate_to(screen):  
    st.session_state.current_screen = screen  
    st.rerun()
```

```
with col1:  
    if st.button("🏠 หน้าแรก", use_container_width=True, key="nav_home"):  
        navigate_to('home')  
  
with col2:  
    if st.button("👤 ประวัติของฉัน", use_container_width=True, key="nav_profile"):  
        navigate_to('profile')  
  
with col3:  
    if st.button("📄 บันทึกสุขภาพ", use_container_width=True, key="nav_health_form"):  
        navigate_to('health_form')  
  
with col4:  
    if st.button("📊 ผลแล็บ", use_container_width=True, key="nav_lab_results"):  
        navigate_to('lab_results')  
  
with col5:  
    if st.button("📈 สรุปผล", use_container_width=True, key="nav_summary"):  
        navigate_to('summary')
```

```
if st.session_state.current_screen == 'home':  
    home_screen()  
elif st.session_state.current_screen == 'profile':  
    profile_screen()  
elif st.session_state.current_screen == 'health_form':  
    health_form_screen()  
elif st.session_state.current_screen == 'lab_results':  
    lab_results_screen()  
elif st.session_state.current_screen == 'summary':  
    summary_screen()
```

Use duckdb

```
query += f"SELECT date {select_columns} from df_user_health_data where 1=1 {filter_year} order by date"  
# print(query)  
result = duckdb.query(query).to_df().to_dict(orient="records")  
return result
```

Use external Database

```
class MyMongoDB():
    def __init__(self, uri):
        self.collection = {}

        # Create a new client and connect to the server
        self.client = MongoClient(uri, server_api=ServerApi('1'))
```

```
self.collection['user_health_data'].update_one(
    {"username": username, "date": date},    # filter
    {
        "$set": {                                # อัปเดตทุกครั้ง
            "health_data": data
        },
        "$setOnInsert": {                        # ตั้งครั้งแรกเท่านั้น
            "username": username,
            "date": date
        }
    },
    upsert=True
)
```

Model Traditional

```
pipeline = ImbPipeline([
    ("preprocessor", preprocessor),
    ("oversample", ros),
    ("model", XGBClassifier(
        objective="multi:softmax",
        eval_metric="mlogloss",
        num_class=num_class,
        random_state=42
    ))
])

# --- Train
pipeline.fit(X_train, y_train)
```

```
def predict_all_models(self, _input_data):
    input_dict = self._get_dict_feature(self.loaded_list_features, _input_data)

    all_predict = {}
    for key in self.loaded_pipelines:
        data = [input_dict[key].values()]

        original_df = pd.read_csv(os.path.join(self.model_dir, f'original_{key}.csv'))

        df_new = pd.DataFrame(data, columns=self.loaded_list_features[key])

        # full pipeline stored in 'model'
        pipeline = self.loaded_pipelines[key]['pipeline']
        le = self.loaded_pipelines[key]['le']

        # Predict class labels (1D array)
        y_pred_new = le.inverse_transform(pipeline.predict(df_new)) # array([1])
        y_prob_new = pipeline.predict_proba(df_new) # array([[0.3, 0.7]]) for example
```

AI Add on

```
@st.cache_resource(ttl=3600)
def init_ai(google_api_key, model_name):
    ai = MyAIGenerator(google_api_key, model_name)
    return ai

@st.cache_resource(ttl=3600)
def get_ai_recommendations(health_data, google_api_key, model_name):
    ai = init_ai(google_api_key, model_name)
    ai_recommendations = ai.get_response(health_data)
    return ai_recommendations
```

Use cache

```
@st.cache_resource(ttl=3600)
def init_model():
    mm = MyModel()
    return mm

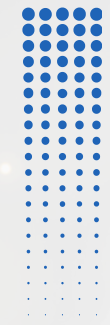
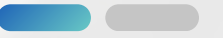
mm = init_model()

@st.cache_resource(ttl=3600)
def get_model_prediction(health_data):
    prediction = mm.predict_all_models(health_data)
    return prediction
```

[Home](#)

[About](#)

[See More](#)



THANK YOU

